Old Exam Questions
DL Architectures & Generative Techniques

Worked Solutions

Stefan Eder

June 24, 2026

How this file is organized

This is the **solutions companion** to `exams.tex`, collected from the main exam (28.06.2024) and the 2022 / 2024 retry exams. Every question is followed *immediately* by a green **answer box**. For **multiple-choice** questions ("select one or more") each option is marked ✓ (correct) or × (incorrect), with a short justification — *why* the correct options are correct and, for the tricky / near-miss distractors, *why* they are wrong. For **calculations** the working is shown; distractor options that exist only to catch a guess get no explanation. If you want a clean practice set with blank answers, use `exams.tex`.

Contents

Image Classification & CNN Architectures

Question 1: AlexNet

Which of the following statements about AlexNet (Krizhevsky et al., 2012) are correct? (Select one or more.)

- ☐ AlexNet does not have fully-connected layers.
- ☐ AlexNet uses strided convolutions instead of max-pooling for down-sampling.
- ☐ AlexNet is a deep convolutional network with more than 1 hidden layer.
- ☐ One of the most important components of AlexNet is BatchNorm.
- ☐ AlexNet uses input images with a resolution of 224×224 pixels and three channels.

Answer: C, E

- ✗ **A.** AlexNet ends in three large *fully-connected* layers — it definitely has them.
- ✗ **B.** It down-samples with *max-pooling* (alongside convolutions); it does not replace pooling with strided convs.
- ✓ **C.** It is a deep CNN — 5 convolutional + 3 fully-connected layers.
- ✗ **D.** BatchNorm did not exist yet (2015). AlexNet's key novelties were ReLU, dropout and local response normalization — not BatchNorm.
- ✓ **E.** Input is $224 \times 224 \times 3$ RGB.

Question 2: VGGNet

Which of the following statements about VGGNet are correct? (Select one or more.)

- ☐ It predominantly uses 3×3 convolutions.
- ☐ It is a single-stage approach (a single 5×5 layer rather than stacked 3×3 layers).
- ☐ It has a symmetric encoder–decoder structure.
- ☐ The majority of the network's weights are located in the fully-connected layers.
- ☐ It cannot be used for image classification.

Answer: A, D

- ✓ **A.** VGG's signature is deep stacks of small 3×3 convolutions.
- ✗ **B.** The opposite: VGG *stacks* two 3×3 layers to match one 5×5's receptive field with fewer parameters and an extra non-linearity.
- ✗ **C.** It is a plain feed-forward classifier, *not* an encoder–decoder.
- ✓ **D.** The huge FC layers (especially the first) hold the bulk of the ∼138M parameters.
- ✗ **E.** Nonsense — VGG is an ImageNet classifier.

Question 3: Normalization in Image Classifiers

Which of the following statements about normalization in image-classification networks are correct? (Select one or more.)

- ☐ Batch Normalization has been used since AlexNet.
- ☐ Since Xception, most image classifiers started to use Layer Normalization.
- ☐ Batch Normalization was used in InceptionNet to speed up training.
- ☐ Batch Normalization is part of most deep networks for image classification (e.g. ResNet).

Answer: C, D

- × **A.** AlexNet used *Local Response Normalization*; BatchNorm only appeared in 2015.
- × **B.** LayerNorm is mainly a transformer / sequence-model thing — CNN classifiers still use BatchNorm.
- ✓ **C.** Inception / GoogLeNet introduced BatchNorm precisely to accelerate training.
- ✓ **D.** BatchNorm is a standard component of deep CNN classifiers such as ResNet.

Question 4: Residual Networks

Which of the following statements about Residual Networks (ResNets) are correct? (Select one or more.)

- ☐ Residual networks use skip-connections that add the representation $\mathbf{h}^l$ of a former layer l to a later layer $l+1$, i.e. $\mathbf{h}^{l+1} = \mathbf{h}^l + F(\mathbf{h}^l)$, where F can be any transformation with appropriate dimensions (e.g. a fully-connected layer).
- ☐ Because of skip-connections, very deep ResNets can be trained even without normalization techniques.
- ☐ Pre-trained ResNets cannot be used for transfer learning because the learned features are usually overly specified to the task on which they were trained.
- ☐ Residual networks are always convolutional and cannot be used in purely fully-connected networks.
- ☐ Leaky ReLUs cannot be used in Residual Networks.

Answer: A

- ✓ **A.** That is exactly the residual formulation; F can be any dimension-matching transform (conv, FC, . . .).
- × **B.** Normalization is still central to training very deep ResNets — skip-connections alone don't remove that need.
- × **C.** Pre-trained ResNets are one of the most common transfer-learning backbones; their features are general, not over-specialised.
- × **D.** Residual connections work in fully-connected nets too — they are not convolution-only.
- × **E.** Any activation, leaky ReLU included, can be used inside a residual block.

Question 5: 1×1 Convolution Output Size

You add a 1×1 convolutional layer with 16 kernels, padding 0, stride 1, to an input of shape $[128, 128, 64]$. Which statement is correct?

- ☐ It is not possible to apply this convolution layer.
- ☐ The resulting dimension is $[128, 128, 16]$.
- ☐ A 1×1 convolution does not exist.
- ☐ Better: replace the 1×1 layer with 2×2 max-pooling — an equivalent but more efficient operation.
- ☐ Better: use 1×1 with stride 2 — equivalent but more efficient.

Answer: B

- × **A.** It is perfectly valid.
- ✓ **B.** A 1×1 conv keeps the spatial size (128×128) and sets the channel depth to the number of kernels (16): $[128, 128, 16]$.
- × **C.** Nonsense — 1×1 convolutions are standard (channel mixing / bottlenecks).
- × **D.** 2×2 max-pooling halves the spatial resolution — not equivalent.
- × **E.** Stride 2 would down-sample to 64×64 — not equivalent.

Question 6: Receptive Field of the Last Layer

You are given a CNN with a fixed architecture. You may only change the parameters of the *last* convolutional layer, and you want to strongly increase the receptive field (RF) of each neuron in that last layer. For each change, decide whether it produces **no increase**, a **small increase**, or a **strong increase** of the RF.

- Increase the stride of the convolution
- Switch to dilated convolutions
- Increase the number of kernels
- Increase the kernel size

Answer: no / strong / no / small

- → **Increase the stride:** *no increase.* Stride in the *last* layer only sub-samples its output; it does not enlarge the input region each of those neurons already sees.
- → **Dilated convolutions:** *strong increase.* Dilation spreads the kernel taps far apart, sharply enlarging the RF.
- → **Number of kernels:** *no increase.* More channels means more features, identical spatial RF.
- → **Kernel size:** *small increase.* A larger kernel adds a few pixels of RF, but only in this one layer.

Question 7: Zero Initialization — Forward Pass

You design a standard 2-layer fully-connected network with sigmoid activations and SGD. You initialize all weights and biases to zero and forward-propagate an input $\mathbf{x}$. What is the output $\hat{y}$?

- ☐ 0
- ☐ e^x
- ☐ 0.5
- ☐ -1
- ☐ 1

Answer: C

- × **A.** Not $\sigma(0)$.
- × **B.** Nonsense.
- ✓ **C.** With all weights and biases zero, every pre-activation is 0, and $\sigma(0) = 0.5$ at every layer — so $\hat{y} = 0.5$.
- × **D.** Out of range for a sigmoid.
- × **E.** Not $\sigma(0)$.

Question 8: Zero Initialization — Weight Update

Consider the same all-zero-initialized network. You forward-propagate a batch, backpropagate, and update the parameters once. $\mathbf{W}^{(2)}$ denotes the weight matrix of the second layer. Which statement is then true?

- ☐ The entries of $\mathbf{W}^{(2)}$ are all positive.
- ☐ The entries of $\mathbf{W}^{(2)}$ may be positive or negative.
- ☐ The entries of $\mathbf{W}^{(2)}$ are all negative.
- ☐ The entries of $\mathbf{W}^{(2)}$ are all zero.

Answer: B

- × **A.** The sign is not forced positive.
- ✓ **B.** The hidden activations are all the same positive value (0.5), but the update sign is set by the per-output error $(\hat{y} - y)$, which can be positive or negative.
- × **C.** The sign is not forced negative.
- × **D.** The near-miss: the second-layer weights *do* get a non-zero gradient because their input (0.5) is non-zero — so they don't stay at zero. (The *first*-layer weights would still be stuck.)

Semantic Segmentation

Question 9: Self-Driving Scene Labelling

A camera in a self-driving car provides road-scene images, and you train a network that outputs an image in which every pixel is annotated as car, road, pedestrian, etc. Which statements are true in this context? (Select one or more.)

- ☐ To train such a network, labels for each pixel of the training images are required.
- ☐ This represents an object-detection task in computer vision.
- ☐ Generative adversarial networks are the most appropriate type of network for this task.
- ☐ Deep networks cannot be used because there is no appropriate loss function.
- ☐ The intersection over union (IoU) would not be an appropriate metric to assess such a model.

Answer: A

- ✓ **A.** Per-pixel labels are needed — this is *semantic segmentation*.
- × **B.** It assigns a class to every pixel, not bounding boxes — not object detection.
- × **C.** A CNN (e.g. U-Net / FCN) with a per-pixel cross-entropy loss is the natural choice, not a GAN.
- × **D.** There is a perfectly good loss (per-pixel cross-entropy) — nonsense.
- × **E.** IoU / mIoU is *the* standard segmentation metric, so it is appropriate.

Question 10: IoU Computation

A ground-truth object covers a set of pixels and a model predicts another set of pixels for that object, on a grid of pixels. The ground-truth region covers 10 pixels, the predicted region covers 8 pixels, and they overlap in 5 pixels. Compute the IoU metric, rounded to three decimals.

Answer: $5/13 \approx 0.385$

$\text{IoU} = \frac{|\text{intersection}|}{|\text{union}|} = \frac{5}{10+8-5} = \frac{5}{13} \approx 0.385$. The union subtracts the double-counted overlap.

Object Detection & Localization

Question 11: Object-Detection Methods

Which of the following statements about deep-learning-based object-detection methods are correct? (Select one or more.)

- ☐ Faster R-CNN uses a region proposal network rather than the selective-search algorithm used in R-CNN.
- ☐ The operations RoI-pooling and RoI-align operate on feature maps and not on the input image.

☐ R-CNN uses AlexNet to extract features for Regions of Interest.

☐ Object detection can be formulated as a regression task in a multi-task setting.

☐ Fast R-CNN has two output layers (heads): one for regression and one for classification.

Answer: A, B, D, E

✓ **A.** Faster R-CNN's contribution is the learned *Region Proposal Network*, replacing selective search.

✓ **B.** Both RoI-pooling and RoI-align pool from the convolutional *feature maps*, not the raw image.

× **C.** R-CNN does run a CNN on each warped proposal, but it is not characterised as "AlexNet" in the lecture — treated as false here.

✓ **D.** Detection is a multi-task problem: box-coordinate *regression* + class prediction.

✓ **E.** Fast R-CNN has a bounding-box regression head and a classification head.

Question 12: One-Stage vs. Two-Stage Detectors

For each architecture, state whether it is a **one-stage** or **two-stage** detector.

- Faster R-CNN
- YOLO
- Mask R-CNN
- Fast R-CNN
- R-CNN
- SSD

Answer: the R-CNN family is two-stage; YOLO and SSD are one-stage

→ Faster R-CNN — **two**-stage (propose regions, then classify/refine)

→ YOLO — **one**-stage

→ Mask R-CNN — **two**-stage

→ Fast R-CNN — **two**-stage

→ R-CNN — **two**-stage

→ SSD — **one**-stage

Rule of thumb: anything in the R-CNN family has a separate proposal stage (two-stage); YOLO/SSD predict boxes and classes in a single pass (one-stage).

Question 13: Transpose Convolution (stride 1)

You are given an input $\mathbf{x} = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}$ and a kernel $\mathbf{w} = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}$. What is the result of the transpose convolution $\mathbf{w} *^T \mathbf{x}$ (stride 1)?

☐ (42)

- ☐ (14)
- ☐ $\begin{pmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & 3 \\ 4 & 4 & 4 & 4 \end{pmatrix}$
- ☐ $\begin{pmatrix} 13 & 13 & 13 \\ 13 & 13 & 13 \\ 13 & 13 & 13 \end{pmatrix}$
- ☐ $\begin{pmatrix} 0 & 2 & 1 \\ 6 & 14 & 6 \\ 2 & 3 & 0 \end{pmatrix}$
- ☐ $\begin{pmatrix} 3 & 2 \\ 1 & 0 \end{pmatrix}$
- ☐ $\begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 2 & 3 \\ 0 & 2 & 0 & 3 \\ 4 & 6 & 6 & 9 \end{pmatrix}$
- ☐ $\begin{pmatrix} 0 & 0 & 1 \\ 0 & 4 & 6 \\ 4 & 12 & 9 \end{pmatrix}$

Answer: H, $\left(\begin{smallmatrix} 0 & 0 & 1 \\ 0 & 4 & 6 \\ 4 & 12 & 9 \end{smallmatrix}\right)$

A stride-1 transpose convolution scatters a copy of the kernel, scaled by each input entry, and overlap-adds them. Output size $(2-1)\cdot 1+2=3$, i.e. 3×3:

$$1\cdot\left(\begin{smallmatrix} 0 & 1 \\ 2 & 3 \end{smallmatrix}\right)\text{ at }(0,1)\ +\ 2\cdot\left(\begin{smallmatrix} 0 & 1 \\ 2 & 3 \end{smallmatrix}\right)\text{ at }(1,0)\ +\ 3\cdot\left(\begin{smallmatrix} 0 & 1 \\ 2 & 3 \end{smallmatrix}\right)\text{ at }(1,1)=\left(\begin{smallmatrix} 0 & 0 & 1 \\ 0 & 4 & 6 \\ 4 & 12 & 9 \end{smallmatrix}\right).$$

The other options are wrong shapes / values that only test whether you actually carried out the overlap-add.

Question 14: Transpose Convolution (stride 2)

You are given an input $\mathbf{x}=\begin{pmatrix} 1 & -2 \\ -1 & 2 \end{pmatrix}$ and a kernel $\mathbf{w}=\begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}$. What is the result of the transpose convolution $\mathbf{w} *^T \mathbf{x}$ with **stride 2**?

- ☐ $\begin{pmatrix} 0 & 1 & -1 \\ 2 & -1 & -1 \\ -4 & -2 & 6 \end{pmatrix}$
- ☐ $\begin{pmatrix} 3 & 2 \\ 1 & 0 \end{pmatrix}$
- ☐ $\begin{pmatrix} 0 & 1 & 0 & -2 \\ 2 & 3 & -4 & -6 \\ 0 & -1 & 0 & 2 \\ -2 & -3 & 4 & 6 \end{pmatrix}$
- ☐ $\begin{pmatrix} 0 & 2 & 1 \\ 6 & -14 & 6 \\ 2 & 3 & 0 \end{pmatrix}$
- ☐ (3)
- ☐ (42)

Answer: C, $\begin{pmatrix} 0 & 1 & 0 & -2 \\ 2 & 3 & -4 & -6 \\ 0 & -1 & 0 & 2 \\ -2 & -3 & 4 & 6 \end{pmatrix}$

With stride 2, the (size-2) kernel copies are placed two apart, so they *don't overlap*; output size $(2-1)\cdot 2+2=4$, i.e. 4×4. Each block is $x_{ij}\cdot\mathbf{w}$ at offset $(2i, 2j)$:

$$\left(\begin{array}{c|c} 1\cdot\left(\begin{smallmatrix} 0 & 1 \\ 2 & 3 \end{smallmatrix}\right) & -2\cdot\left(\begin{smallmatrix} 0 & 1 \\ 2 & 3 \end{smallmatrix}\right) \\ \hline -1\cdot\left(\begin{smallmatrix} 0 & 1 \\ 2 & 3 \end{smallmatrix}\right) & 2\cdot\left(\begin{smallmatrix} 0 & 1 \\ 2 & 3 \end{smallmatrix}\right) \end{array}\right) = \begin{pmatrix} 0 & 1 & 0 & -2 \\ 2 & 3 & -4 & -6 \\ 0 & -1 & 0 & 2 \\ -2 & -3 & 4 & 6 \end{pmatrix}.$$

The remaining options are wrong sizes / values to catch a guess.

Autoencoders

Question 15: Annotating the Generative-Model Taxonomy

A figure shows three categories of generative models. Each row is a network with grey input/output boxes and a latent variable $\mathbf{z}$ in the middle. Annotate the building blocks with *Generator*, *Discriminator*, *Encoder*, *Decoder*, *Flow*, *Inverse Flow*, given that a rectangular block keeps the dimension constant while a trapezoidal block decreases or increases it.

- Top row: input → trapezoid → 0/1 → $\mathbf{z}$ → trapezoid → output
- Middle row: input → trapezoid (down) → $\mathbf{z}$ → trapezoid (up) → output
- Bottom row: input → rectangle → $\mathbf{z}$ → rectangle → output

Answer: GAN / (V)AE / Flow

→ **Top row = GAN.** The block ending in a 0/1 decision is the **Discriminator**; the block going from $\mathbf{z}$ to the output is the **Generator**.

→ **Middle row = (V)AE.** The down-sizing trapezoid is the **Encoder** $q(\mathbf{z}\,|\,\mathbf{x})$; the up-sizing one is the **Decoder** $p(\mathbf{x}\,|\,\mathbf{z})$.

→ **Bottom row = normalizing flow.** Dimensions are preserved (rectangles): the first block is the **Flow**, the second its **Inverse Flow**.

Question 16: Autoencoders (excluding VAEs)

Which of the following statements about autoencoders (**not** including variational autoencoders) are correct? (Select one or more.)

☐ The input–output Jacobian of an autoencoder is always a lower-triangular matrix.

☐ Autoencoders do not map a single input to a single point in latent space, but to a distribution in latent space.

☐ Autoencoders are always fully-connected networks.

☐ All autoencoders use fewer neurons in the hidden layers than in the input layer.

☐ Autoencoders consist of an encoder and a decoder, which are parametrized neural networks.

Answer: E

- ✗ **A.** There is no such constraint — the Jacobian is generally not triangular.
- ✗ **B.** Mapping an input to a *distribution* is the VAE; a plain AE maps it to a single latent *point*.
- ✗ **C.** They can be convolutional, recurrent, etc. — not necessarily fully-connected.
- ✗ **D.** Undercomplete is common but not required: overcomplete / sparse AEs have $\geq$ input-size hidden layers.
- ✓ **E.** The only always-true statement: an AE is an encoder + decoder, both parametrized networks.

Question 17: Sparse and Denoising Autoencoders

Which of the following statements about sparse / denoising autoencoders are correct? (Select one or more.)

- ☐ Sparse AEs enforce sparsity on the representation via a regularization term.
- ☐ Sparse AEs enforce hidden units to be inactive (i.e. activation $= 0$) for most inputs.
- ☐ A sparse AE can be trained on a copy task, where the input equals the target.
- ☐ Sparse AEs force the *input* units to be inactive (activation $= 0$).
- ☐ Sparse AEs enforce sparsity via early stopping.
- ☐ A sparse AE cannot be trained on a denoising task (input noisy, target clean).

Answer: A, B, C

- ✓ **A.** Sparsity is added as a regularization term (e.g. a KL penalty on mean activation, or L_1).
- ✓ **B.** The effect is that most *hidden* units are (near-)zero for any given input.
- ✓ **C.** It can be trained on the ordinary copy / reconstruction task (input = target).
- ✗ **D.** It silences *hidden* units, not input units — the input is the data.
- ✗ **E.** Sparsity comes from the penalty term, not from early stopping.
- ✗ **F.** Nothing stops combining a sparsity penalty with a denoising objective.

Question 18: Autoencoder Reconstruction Loss

An autoencoder uses ReLU activations with encoder $u = \mathrm{ReLU}\left(\begin{bmatrix}1 & 0 & -1\\ 0 & 1 & 0\end{bmatrix}\mathbf{x}\right)$ and decoder $\hat{\mathbf{x}} = \mathrm{ReLU}\left(\begin{bmatrix}1 & 0\\ 0 & 0\\ 1 & 0\end{bmatrix} u\right)$. Compute the reconstruction loss $\mathcal{L}(\mathbf{x}, \hat{\mathbf{x}}) = \sum_i (x_i - \hat{x}_i)^2$ for the input $\mathbf{x} = (1, 1, 2)^\mathsf{T}$.

Answer: 6

Encoder: $\begin{bmatrix}1 & 0 & -1\\ 0 & 1 & 0\end{bmatrix}(1,1,2)^\mathsf{T} = (1-2,\ 1) = (-1, 1)$, so $u = \mathrm{ReLU}(-1, 1) = (0, 1)$.

Decoder: $\begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 1 & 0 \end{bmatrix} (0,1)^\mathsf{T} = (0,0,0)$, so $\hat{\mathbf{x}} = \mathrm{ReLU}(0,0,0) = (0,0,0)$.
Loss $= (1-0)^2 + (1-0)^2 + (2-0)^2 = 1+1+4 = 6$. (If a $\frac{1}{n}$ MSE were used instead of the sum, the value would be 2.0.)

Variational Autoencoders

Question 19: Variational Autoencoders

Which of the following statements about variational autoencoders (VAEs) are true? (Select one or more.)

- ☐ VAEs have an encoder–decoder structure, where encoder and decoder must be symmetric.
- ☐ The prior distribution of the latent variables $\mathbf{z}$ must be a discrete distribution.
- ☐ VAEs maximize a lower bound on the likelihood of the data.
- ☐ VAEs automatically provide the marginal likelihood $p(\mathbf{x})$ at low computational cost.
- ☐ VAEs cannot be used to generate images.

Answer: C

- × **A.** Encoder and decoder need not be symmetric.
- × **B.** The prior is typically *continuous* (a standard Gaussian), not discrete.
- ✓ **C.** VAEs maximize the ELBO, a lower bound on the data log-likelihood.
- × **D.** They do *not* give $p(\mathbf{x})$ cheaply — the marginal stays intractable; the ELBO is only a bound.
- × **E.** Nonsense — generating images is a primary use of VAEs.

Question 20: VAE Objective Function

Using the notation from the lecture, which of the following is the correct formulation of the standard Variational Autoencoder (VAE) objective?

- ☐ $\mathcal{L}(D,G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[\log(1 - D(G(\mathbf{z})))]$
- ☐ $\mathcal{L}_{\theta,\phi}(\mathbf{x}) = \mathbb{E}_{q_\phi(\mathbf{z}\,|\,\mathbf{x})} \log p_\theta(\mathbf{x}\,|\,\mathbf{z}) - D_{\mathrm{KL}}(p_\theta(\mathbf{x}) \,\|\, p_\theta(\mathbf{z}))$
- ☐ $\mathcal{L}(D,G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[D(G(\mathbf{z}))]$
- ☐ $\mathcal{L}_{\theta,\phi}(\mathbf{x}) = \mathbb{E}_{q_\phi(\mathbf{z}\,|\,\mathbf{x})} \log p_\theta(\mathbf{x}\,|\,\mathbf{z}) - D_{\mathrm{KL}}(q_\phi(\mathbf{z}\,|\,\mathbf{x}) \,\|\, p_\theta(\mathbf{z}))$
- ☐ $\mathcal{L}_\theta(\mathbf{x}) = D_{\mathrm{KL}}(p_{\mathrm{data}}(\mathbf{x};\theta) \,\|\, p^*(\mathbf{x}))$

Answer: D

- × **A.** This is the GAN min–max value function, not the VAE objective.
- × **B.** Right shape, *wrong KL arguments*: the KL must be between $q_\phi(\mathbf{z}\,|\,\mathbf{x})$ and the prior $p_\theta(\mathbf{z})$, not between $p_\theta(\mathbf{x})$ and $p_\theta(\mathbf{z})$.

× **C.** Another GAN-style objective (and missing the log on the second term).

✓ **D.** The ELBO: reconstruction term $\mathbb{E}_{q_\phi(\mathbf{z}\,|\,\mathbf{x})} \log p_\theta(\mathbf{x}\,|\,\mathbf{z})$ minus $D_{\mathrm{KL}}(q_\phi(\mathbf{z}\,|\,\mathbf{x}) \,\|\, p_\theta(\mathbf{z}))$.

× **E.** Nonsense — not the VAE objective.

Question 21: Best Gaussian Approximation via KL

Asked multiple times.

You have a fixed, multi-dimensional, non-Gaussian distribution $p(\mathbf{x})$ that you want to approximate with a Gaussian $q(\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, by minimizing $D_{\mathrm{KL}}(p \,\|\, q)$ with respect to $\boldsymbol{\mu}$ and $\boldsymbol{\Sigma}$. Which expressions minimize the KL-divergence? ($\mathbb{E}$ is expectation, Cov the covariance, $\mathbf{I}$ the identity matrix.)

☐ $\boldsymbol{\mu} = \mathbb{E}(\mathbf{x})$ and $\boldsymbol{\Sigma} = \mathrm{Cov}(\mathbf{x})$

☐ $\boldsymbol{\mu} = \mathbb{E}(\mathbf{x})$ and $\boldsymbol{\Sigma} = \mathbf{I}$

☐ $\boldsymbol{\mu} = \mathbb{E}(\mathbf{x}^\mathsf{T}\mathbf{x})$ and $\boldsymbol{\Sigma} = \mathbb{E}(\mathbf{x}^\mathsf{T})$

☐ $\boldsymbol{\mu} = \mathbb{E}(\mathbf{x}^\mathsf{T}\mathbf{x})$ and $\boldsymbol{\Sigma} = \mathbf{I}$

☐ $\boldsymbol{\mu} = \mathbb{E}(\mathbf{x})$ and $\boldsymbol{\Sigma} = \mathbb{E}(\mathbf{x}\mathbf{x}^\mathsf{T})$

Answer: A

✓ **A.** Minimizing $D_{\mathrm{KL}}(p \,\|\, q)$ over a Gaussian *matches the first two moments*: $\boldsymbol{\mu} = \mathbb{E}(\mathbf{x})$, $\boldsymbol{\Sigma} = \mathrm{Cov}(\mathbf{x})$.

× **B.** Forcing $\boldsymbol{\Sigma} = \mathbf{I}$ throws away the true covariance.

× **C.** $\boldsymbol{\mu} = \mathbb{E}(\mathbf{x}^\mathsf{T}\mathbf{x})$ is a scalar — dimensionally nonsense.

× **D.** Same dimensional nonsense for $\boldsymbol{\mu}$.

× **E.** Near-miss: $\mathbb{E}(\mathbf{x}\mathbf{x}^\mathsf{T})$ is the *second moment*, which over-estimates the covariance because it omits the $-\mathbb{E}(\mathbf{x})\mathbb{E}(\mathbf{x})^\mathsf{T}$ correction.

GANs and Wasserstein GANs

Question 22: GAN Objective Function

Using the lecture notation, which of the following is the correct formulation of the standard objective of Generative Adversarial Networks (GANs)?

☐ $\min_G \max_D \mathcal{L}(D, G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[\log(1 - D(\mathbf{z}))]$

☐ $\min_G \max_D \mathcal{L}(D, G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{x}\sim p_r}[\log(1 - D(G(\mathbf{x})))]$

☐ $\min_G \max_D \mathcal{L}(D, G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log G(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[\log(1 - G(D(\mathbf{z})))]$

☐ $\min_G \max_D \mathcal{L}(D, G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[\log(1 - D(G(\mathbf{z})))]$

☐ $\min_G \max_D \mathcal{L}(D, G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[D(G(\mathbf{z}))]$

☐ $\min_G \max_D \mathcal{L}(D, G) = \mathbb{E}_{\mathbf{x}\sim p_r}[\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z}\sim p_z}[\log(D(G(\mathbf{z})))]$

Answer: D

- ✗ **A.** The fake term must pass through G: $D(G(\mathbf{z}))$, not $D(\mathbf{z})$.
- ✗ **B.** The second expectation must be over $\mathbf{z} \sim p_z$, not over real data $\mathbf{x} \sim p_r$.
- ✗ **C.** Roles of D and G are swapped and the wrong functions are logged — nonsense.
- ✓ **D.** Correct value function: $\mathbb{E}_{p_r}[\log D(\mathbf{x})] + \mathbb{E}_{p_z}[\log(1 - D(G(\mathbf{z})))]$.
- ✗ **E.** Missing the $\log(1 - \cdot)$ on the fake term.
- ✗ **F.** Wrong form: should be $\log(1 - D(G(\mathbf{z})))$, not $\log(D(G(\mathbf{z})))$.

Question 23: Properties of a Trained GAN

Consider a GAN that successfully produces images of apples. Which statements are correct? (Select one or more.)

- ☐ The discriminator can be used to classify images as apple vs. non-apple.
- ☐ The generator aims to learn the distribution of apple images.
- ☐ The generator can produce unseen images of apples.
- ☐ After training the GAN, the discriminator loss eventually reaches a constant value.

Answer: B, C

- ✗ **A.** The discriminator only learns *real-vs-fake* apples, not apple-vs-non-apple.
- ✓ **B.** The generator's goal is exactly to model the distribution of apple images.
- ✓ **C.** A good generator generalizes and produces novel, unseen apples.
- ✗ **D.** Not guaranteed: at the ideal equilibrium $D = 0.5$ the loss would be constant, but in general GAN training need not converge there.

Question 24: Divergences and Distances

Which of the following statements about divergences and distances are correct? (Select one or more.)

- ☐ JS and KL divergences show problems for distributions with disjoint supports — the Wasserstein distance overcomes this.
- ☐ A map $D : \mathcal{S} \times \mathcal{S} \to \mathbb{R}$ is a divergence iff it is non-negative and symmetric.
- ☐ The Wasserstein-1 distance with Euclidean cost is a distance, not a divergence.
- ☐ The Wasserstein distance is defined as a minimization over joint distributions (couplings), but this is infeasible in practice.
- ☐ The Kantorovich–Rubinstein theorem lets us write the Wasserstein distance as a maximization over Lipschitz functions, which makes it tractable.

Answer: A, C, D, E

- ✓ **A.** KL / JS blow up or stay constant for disjoint supports; Wasserstein still gives a finite, smooth value.
- ✗ **B.** A divergence need *not* be symmetric — non-negative + symmetric (with the triangle

inequality) describes a *distance/metric*, not a divergence.

✓ **C.** W_1 with Euclidean cost satisfies the metric axioms (symmetry, triangle inequality), so it is a genuine distance.

✓ **D.** It is an infimum over couplings (joint distributions) — intractable to compute directly.

✓ **E.** Kantorovich–Rubinstein duality rewrites it as a supremum over 1-Lipschitz functions — exactly what WGAN exploits.

Question 25: KL, JSD and Wasserstein vs. Separation

Let $p_1 = \mathcal{N}(0, 1)$ and $p_2 = \mathcal{N}(x, 1)$. As x increases, you plot the KL-divergence $D_{\mathrm{KL}}(p_1 \,\|\, p_2)$, the Jensen–Shannon divergence $\mathrm{JSD}(p_1 \| p_2)$, and the Wasserstein-2 distance $\mathrm{WD}(p_1 \| p_2)$ against x. One curve grows quadratically without bound, one grows linearly, and one saturates at a finite value. Match each measure to its qualitative behaviour.

- KL-divergence
- Wasserstein-2 distance
- Jensen–Shannon divergence

Answer: KL quadratic, W_2 linear, JSD saturates

→ **KL-divergence:** grows *quadratically* without bound — for equal-variance Gaussians $D_{\mathrm{KL}}(p_1 \,\|\, p_2) = x^2/2$.

→ **Wasserstein-2:** grows *linearly* — $\mathrm{WD}_2 = |x|$.

→ **Jensen–Shannon:** *saturates* at a finite value (bounded by $\log 2$) once the supports barely overlap.

This is why JS/KL give vanishing gradients for far-apart distributions while Wasserstein stays informative.

Metrics, Normalizing Flows & Density Models

Question 26: Inception Score and FID

Tick the correct statements about the Inception Score (IS) and the Fréchet Inception Distance (FID). (Select one or more.)

☐ FID can detect mode collapse, because mode collapse influences the covariance matrix of the generated data.

☐ IS and FID are metrics that assess the quality of GAN-generated images.

☐ Both IS and FID rely on features of the input images produced by AlexNet.

☐ IS and FID can be used as GAN objectives that lead to faster training.

☐ The computation of the FID is based on an explicit formula for the Wasserstein-2 distance between two Gaussian distributions.

Answer: A, B, E

- ✓ **A.** FID compares means *and covariances*; mode collapse shrinks the covariance, so FID picks it up.
- ✓ **B.** Both are standard metrics for the quality of GAN-generated images.
- × **C.** They use *Inception* (v3) features, not AlexNet.
- × **D.** They are *evaluation* metrics, not training objectives.
- ✓ **E.** FID is exactly the closed-form Wasserstein-2 (Fréchet) distance between two Gaussians fitted to the features.

Question 27: Normalizing Flows

Which of the following statements about normalizing flows are correct? (Select one or more.)

- ☐ It is impossible to integrate a multi-layer perceptron as a transformation in a normalizing flow because MLPs are not invertible.
- ☐ All multi-layer perceptrons with the same input and output dimension can be considered a normalizing flow.
- ☐ Normalizing flows cannot contain functions with a lower-triangular Jacobian because they are not invertible.
- ☐ Some normalizing flows use functions with a lower-triangular Jacobian to facilitate the computation of the determinant.

Answer: D

- × **A.** "Impossible" is too strong — one *can* design invertible MLP-based transforms; ordinary MLPs simply aren't invertible by default.
- × **B.** Matching input/output dimensions does not make a network invertible / a valid flow.
- × **C.** Lower-triangular-Jacobian maps can absolutely be invertible — that is the whole point of autoregressive flows.
- ✓ **D.** A triangular Jacobian makes the determinant the product of its diagonal — cheap to compute.

Question 28: Flow-Based Models

Which of the following statements about flow-based models are correct? (Select one or more.)

- ☐ Theoretically, flow-based models can represent any distribution $p_x(\mathbf{x})$ if one starts from a sufficiently complex distribution; a simple base distribution $p_u(\mathbf{u})$ such as the uniform on the cube $(0, 1)^D$ would not be sufficient.
- ☐ Fitting flow-based models is often done with the KL-divergence.
- ☐ Computing the Jacobi determinant of a neural network with D inputs and D outputs is in general of $\mathcal{O}(D^3)$.
- ☐ For autoregressive flows the transformation g_ϕ is constructed so that its Jacobi determinant always has value 1.

Answer: B, C

- × **A.** A *simple* base (even uniform on $(0,1)^D$ or a standard Gaussian) is sufficient — the flexible transform does the work.
- ✓ **B.** Flows are fit by maximum likelihood, i.e. minimizing a KL-divergence to the data.
- ✓ **C.** A general dense Jacobian determinant costs $\mathcal{O}(D^3)$.
- × **D.** An autoregressive flow's Jacobian is triangular, but its determinant (product of the diagonal) is *not* fixed at 1.

Question 29: Density under a Linear Transform

Asked multiple times.

Assume a two-dimensional vector $\mathbf{u} \sim p_u(\mathbf{u})$. Let $T = \begin{pmatrix} 2 & 0 \\ 0 & \frac{1}{2} \end{pmatrix}$ and $\mathbf{x} = T \cdot \mathbf{u}$. Mark the correct formula for the distribution $p_x(\mathbf{x})$.

- ☐ $p_u\left(\begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 2 \end{pmatrix} \mathbf{x}\right) \cdot \frac{1}{2}$
- ☐ $p_u\left(\begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 2 \end{pmatrix} \mathbf{x}\right)$
- ☐ $p_u\left(\begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 1 \end{pmatrix} \mathbf{x}\right) \cdot \frac{1}{2}$
- ☐ $p_u\left(\begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 1 \end{pmatrix} \mathbf{x}\right)$
- ☐ $p_u\left(\begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \mathbf{x}\right) \cdot 2$
- ☐ $p_u\left(\begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \mathbf{x}\right) \cdot \frac{1}{2}$

Answer: B

- ✓ **B.** Change of variables: $p_x(\mathbf{x}) = \frac{1}{|\det T|}\, p_u(T^{-1}\mathbf{x})$ with $T^{-1} = \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 2 \end{pmatrix}$ and $\det T = 2 \cdot \frac{1}{2} = 1$, so the prefactor is 1.
- × **A.** Correct T^{-1} but a spurious $\frac{1}{2}$ factor ($\det T = 1$, not 2).
- × **C–F.** Wrong inverse matrix and/or wrong determinant prefactor — guess-catchers.

Question 30: Autoregressive Sequence Probability

An autoregressive model $p_w(\mathbf{x})$ models sequences (x_1, x_2, x_3) where each element is A, B, or C:

$$p_w(x_1) = \tfrac{1}{3}, \qquad p_w(x_2 \mid x_1) = \begin{cases} \frac{1}{2} & x_2 = x_1 \\ \frac{1}{4} & x_2 \neq x_1 \end{cases}, \qquad p_w(x_3 \mid x_2, x_1) = \begin{cases} \frac{1}{2} & x_3 = x_2 \\ \frac{1}{4} & x_3 \neq x_2 \end{cases}.$$

Compute $p_w(\mathbf{x})$ for $\mathbf{x} = (A, A, C)$, rounded to three decimals.

Answer: $1/24 \approx 0.042$

$$p_w(A, A, C) = \underbrace{p_w(x_1{=}A)}_{1/3} \cdot \underbrace{p_w(x_2{=}A \mid A)}_{1/2\ (x_2=x_1)} \cdot \underbrace{p_w(x_3{=}C \mid A)}_{1/4\ (x_3\neq x_2)} = \tfrac{1}{3} \cdot \tfrac{1}{2} \cdot \tfrac{1}{4} = \tfrac{1}{24} \approx 0.042.$$

Bayesian Deep Learning

Question 31: Deep Ensembles and Marginalization

The main difference between the frequentist and Bayesian approach is the marginalization over parameters when defining the probability of a class label given input $\mathbf{x}$ and dataset $\mathcal{D}$:

$$p(\mathbf{y}^{\text{test}} \mid \mathbf{x}^{\text{test}}, \mathcal{D}) = \mathbb{E}_{\mathbf{w}\sim p(\mathbf{w}\mid\mathcal{D})}\left[p(\mathbf{y}^{\text{test}} \mid \mathbf{w}, \mathbf{x}^{\text{test}})\right].$$

Deep Ensembles approximate this full expectation with (weighted) averages. Which of the following are valid approximations? (Select one or more.)

- ☐ $\frac{1}{M}\sum_{m=1}^{M} p(\mathbf{y}^{\text{test}} \mid \mathbf{w}_m, \mathbf{x}^{\text{test}})$, where $\mathbf{w}_m$ are the weights of the m-th network of the ensemble.
- ☐ $p(\mathbf{y}^{\text{test}} \mid \mathbf{w}, \mathbf{x}^{\text{test}})$ for a single trained network.
- ☐ $\frac{1}{M}\sum_{m=1}^{M} \alpha_m\, p(\mathbf{y}^{\text{test}} \mid \mathbf{w}_m, \mathbf{x}^{\text{test}})$ with $\alpha_m = \text{softmax}(\mathbf{z})_m$ for some random vector $\mathbf{z}$.
- ☐ $\frac{1}{M}\sum_{m=1}^{M} p(\mathbf{y}_m^{\text{test}} \mid \mathbf{w}, \mathbf{x}_m^{\text{test}})$, where m denotes the m-th equally-sized fraction of the test data.

Answer: A, C

- ✓ **A.** A uniform average over the M ensemble members approximates the posterior expectation.
- × **B.** A single network is just one sample — no marginalization over the posterior.
- ✓ **C.** A *weighted* average (weights from a softmax, summing to 1) is also a valid approximation of the expectation.
- × **D.** This splits the *test data* into fractions — that is not averaging over parameters at all.

Graph Neural Networks

Question 32: Distinguishing Two Small Graphs

Consider two graphs: $\mathcal{G}_1$ is a path of three nodes (a coloured centre node joined to two leaves), and $\mathcal{G}_2$ is a star with a centre node joined to three leaves. Mark the correct statements. (Select one or more.)

- ☐ $\mathcal{G}_1$ and $\mathcal{G}_2$ can be distinguished by the Weisfeiler–Lehman (colour refinement) test.
- ☐ All message-passing neural networks (MPNNs) can distinguish $\mathcal{G}_1$ and $\mathcal{G}_2$.
- ☐ $\mathcal{G}_1$ and $\mathcal{G}_2$ are isomorphic graphs.

Answer: A

✓ **A.** They have different structure (different node counts / degree sequences), so colour refinement separates them.

× **B.** "All MPNNs" is too strong — expressivity depends on the aggregation; a weak MPNN need not distinguish arbitrary non-isomorphic graphs.

× **C.** They are not isomorphic (different structure).

Question 33: Weisfeiler–Lehman Necessary Condition

Two graphs A and B are given. Do they fulfil the necessary condition for graph isomorphism given by the Weisfeiler–Lehman test (i.e. do they produce the same stable colour histogram)?

☐ True

☐ False

Answer: True

✓ **True.** The two graphs produce the same stable WL colour histogram, so the *necessary* condition for isomorphism is satisfied. (Necessary, not sufficient — equal histograms don't prove isomorphism.)

× **False.** —

Question 34: Matching Graphs to Adjacency Matrices

Four graphs on six nodes are shown: A is a plain 6-cycle (hexagon), B is two triangles, C is another 6-cycle drawn differently, and D is a 6-cycle with one extra chord. For each adjacency matrix, name the matching graph (A, B, C, D, or none).

- All six row-sums equal 2 and the edges form a single closed cycle.
- Two row-sums equal 1 and the rest equal 2 (an open path, not a cycle).
- All six row-sums equal 2 but the edges split into two separate 3-cycles.
- Four row-sums equal 2 and two equal 3 (a cycle plus one chord).

Answer: A/C / **none** / B / D

Read off each matrix by its row-sums (node degrees) and connectivity:

→ All degrees 2, one closed cycle ⇒ a 6-cycle: A (and the differently-drawn cycle is C).

→ Two degree-1 nodes (open path) ⇒ **none of the above** (no shown graph is an open path).

→ All degrees 2 but split into two 3-cycles ⇒ B (two triangles).

→ Two degree-3 nodes (cycle + chord) ⇒ D.

Question 35: Group Action in Graph NNs

Which of the following is a common group action used in graph neural networks?

☐ Rotation

- ☐ Scaling
- ☐ Translation (permutation / relabelling of nodes)
- ☐ Cropping
- ☐ Clustering

Answer: C

- × **A.** Image augmentation, not the GNN symmetry.
- × **B.** Image augmentation, not relevant here.
- ✓ **C.** GNNs are built to be invariant / equivariant to *permutations* (relabelling) of the nodes.
- × **D.** Image augmentation.
- × **E.** A downstream task, not a group action.

Question 36: Equivariance of an Attention-Pooling Layer

A layer implements $\mathbf{h}^{(l+1)} = M \underbrace{\mathrm{softmax}(\beta\, M^{\mathsf{T}}\mathbf{h}^{(l)})}_{=:\mathbf{p}}$, where $\mathbf{h}^{(l)} \in \mathbb{R}^d$, $\beta > 0$ is a hyperparameter, and $M \in \mathbb{R}^{d\times J}$ holds J pattern columns. Which statements about this layer are true? (Select one or more.)

- ☐ $\mathbf{h}^{(l+1)}$ is equivariant to permutations of the columns of M.
- ☐ $\mathbf{p}$ is invariant to permutations of the columns of M.
- ☐ $\mathbf{p}$ is equivariant to permutations of the columns of M.
- ☐ $\mathbf{h}^{(l+1)}$ is invariant to permutations of the columns of M.
- ☐ $\mathbf{p}$ is equivariant to permutations of the rows of M.
- ☐ This layer cannot be used in deep learning because the gradient $\partial\mathbf{h}^{(l+1)}/\partial\mathbf{h}^{(l)}$ cannot be backpropagated.

Answer: C, D

- × **A.** $\mathbf{h}^{(l+1)}$ is *invariant*, not equivariant, to column permutations of M.
- × **B.** $\mathbf{p}$'s entries permute under column permutations of M, so it is not invariant.
- ✓ **C.** Permuting the columns of M permutes the rows of M^{T}, hence permutes the entries of $\mathbf{p}$ — $\mathbf{p}$ is *equivariant*.
- ✓ **D.** $\mathbf{h}^{(l+1)} = M\mathbf{p}$ is unchanged: the column reorder of M cancels the row reorder of $\mathbf{p}$ — so it is *invariant*.
- × **E.** Row permutations act on the feature dimension d, not on the J pattern entries of $\mathbf{p}$ — this does not give equivariance of $\mathbf{p}$.
- × **F.** The layer (softmax + matrix products) is differentiable — gradients backpropagate fine.

Theory & Foundations

Question 37: Universal Approximator Theorem

Mark the correct statements about the Universal Approximator Theorem (UAT) for neural networks. (Select one or more.)

- ☐ For a real-valued, continuous function h on $[-\pi, \pi]^2$, the existence of an approximating network can be derived with the help of Fourier series.
- ☐ The UAT is a theoretical statement about the expressive power of neural networks.
- ☐ The UAT gives a useful algorithm to compute the optimal weights of a network that should approximate a given continuous function.
- ☐ The algorithm related to the UAT is easy to understand and implement, but very costly and therefore hardly used in practice.
- ☐ The UAT shows that any function can be approximated by any given network with arbitrary precision; there are no restrictions on the function or on the network structure.

Answer: A, B

- ✓ **A.** One proof route builds the approximation from Fourier series on the compact domain.
- ✓ **B.** The UAT is purely an existence / expressivity statement.
- × **C.** It is *non-constructive* — it does not tell you how to find the weights.
- × **D.** There is no practical "UAT algorithm" in use — nonsense.
- × **E.** There are conditions: the target must be continuous on a compact set and the network needs at least one (sufficiently wide) hidden layer — "any function, no restrictions" is false.

Question 38: Newton's Method vs. Gradient Direction

On a quadratic error surface (elliptical level curves), two arrows are drawn from a point: one points straight at the minimum at the centre of the ellipses, the other points perpendicular to the local level curve. Label each arrow as *Gradient* or *Newton's Method*.

- Arrow pointing directly to the centre (minimum)
- Arrow perpendicular to the level curve (steepest descent)

Answer: centre = Newton, perpendicular = Gradient

- → **Arrow to the centre = Newton's Method.** It pre-multiplies the gradient by the inverse Hessian, so on a quadratic it points straight at the minimum in one step.
- → **Arrow perpendicular to the level curve = Gradient.** Steepest descent is orthogonal to the contour, which generally does *not* aim at the minimum on elongated ellipses.